

TECHNICAL REPORT

FinDocIQ

A Sovereign Document-Intelligence Pipeline
for Financial Documents

Author:
Amina Ouaayi

Supervisor:
Abderrahmane Samali

Academic Year 2025–2026

Abstract

FinDocIQ is an end-to-end pipeline for extracting structured information from, and answering natural-language questions over, financial documents (invoices, statements, annual reports). The system supports both digital PDFs and scanned images, runs entirely on self-hosted infrastructure (Ollama for language models, RapidOCR/DeepSeek-OCR for optical character recognition), and produces grounded answers with visible source citations. This report documents the system architecture, the sovereignty rationale behind the design choices, and an honest, quantitative evaluation of the pipeline's performance — including a direct discussion of its failure modes, in particular hallucination behaviour and multi-step arithmetic reasoning.

Contents

1	Introduction	3
2	System Architecture	3
2.1	Design Principle: Heavy Work Runs Once, the App Stays Light	3
2.2	Pipeline Overview	3
2.3	Component Summary	3
3	Data Sovereignty Rationale	3
4	Evaluation Methodology and Results	4
4.1	Structured Extraction	4
4.2	Grounded Question Answering and RAG	5
4.3	OCR Accuracy	5
4.4	Application Interface	7
4.5	Automated Financial Consistency Checks	8
5	Hallucination and Guardrail Discussion	8
5.1	Guardrail Design and Measurement	8
5.2	Retrieval Tuning: $k = 3$ versus $k = 5$	9
5.3	The Real Bottleneck: Arithmetic Reasoning, Not Retrieval	10
6	Limitations	10
7	Gold Set Construction	11
8	Conclusion and Future Work	11

1 Introduction

Financial documents — invoices, bank statements, annual reports — carry information that is both highly structured (amounts, dates, identifiers) and embedded in free-form text and tables. Manually locating and cross-referencing this information does not scale. FinDocIQ addresses this with two complementary capabilities:

1. **Structured extraction:** turning a document into a clean JSON record (issuer, date, total, line items).
2. **Grounded question answering:** allowing a user to ask a free-form question and receive an answer that cites its exact source, over one document or a whole collection.

2 System Architecture

2.1 Design Principle: Heavy Work Runs Once, the App Stays Light

The central architectural decision is a separation between an offline, compute-heavy *extraction* stage and a lightweight, stateless *application* stage. Extraction (OCR, embedding, LLM calls) runs once per document and its output is cached; the Streamlit application only ever reads cached JSON or performs a small number of retrieval and generation calls at query time. This is why the system requires no GPU on the client machine, and it mirrors how production document-intelligence systems are typically structured.

2.2 Pipeline Overview

Figure 1 shows the full pipeline. An incoming document is first routed based on whether it carries a digital text layer. Digital PDFs are parsed directly; scanned documents are passed through OCR (RapidOCR locally on CPU, with hard cases intended to route to a server-hosted DeepSeek-OCR model). The resulting text feeds a structured-extraction step, and separately, one of two question-answering modes: a single-document direct path, or a multi-document Retrieval-Augmented Generation (RAG) path.

2.3 Component Summary

Table 1 summarises the concrete technology choice for each layer of the pipeline and the reasoning behind it.

3 Data Sovereignty Rationale

Financial documents are sensitive by nature: they may contain client identities, transaction amounts, and business relationships that should not be exposed to third parties. This constraint shaped the architecture from the first day of the project, not as an afterthought:

- **No cloud LLM APIs are used.** All language-model inference runs through Ollama, either on the local development machine or on the team’s self-hosted server. Document text is never transmitted to an external provider.
- **OCR follows the same principle.** RapidOCR runs entirely on CPU, locally. The heavier DeepSeek-OCR model, reserved for difficult scans, is likewise hosted on team infrastructure rather than a commercial OCR API.

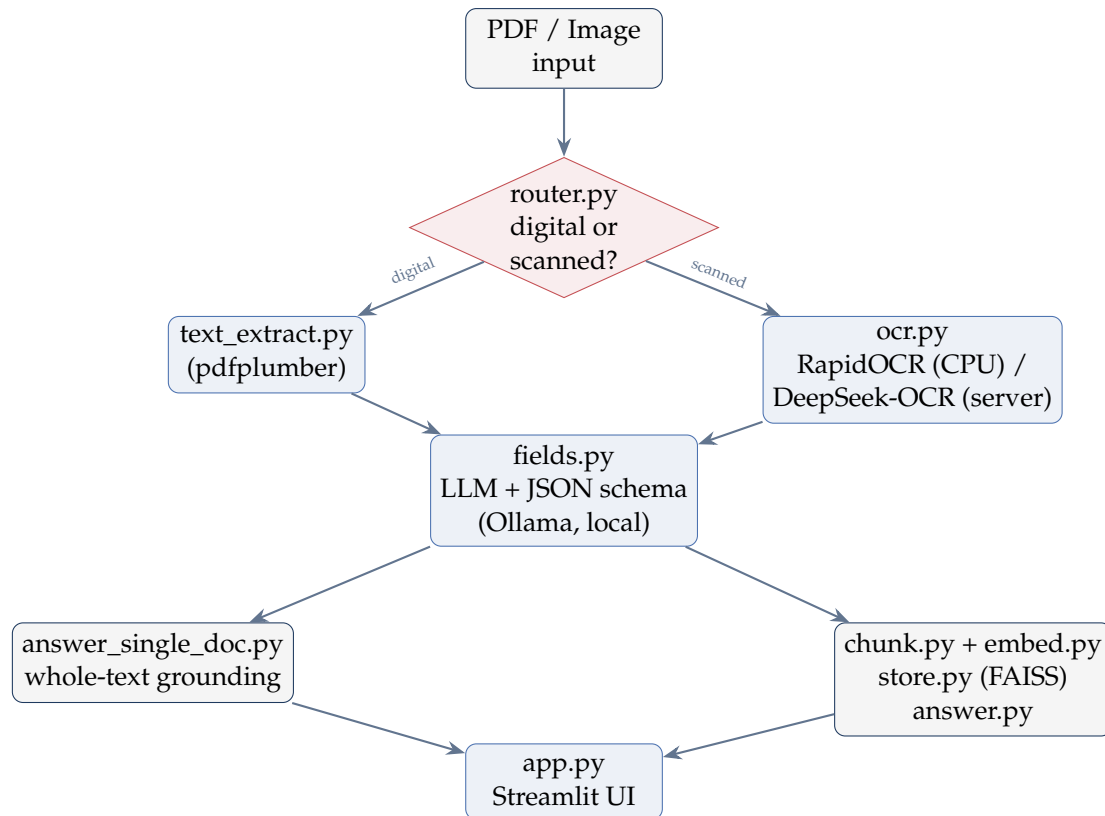


Figure 1: FinDocIQ pipeline. Extraction (top) runs once per document; the two Q&A modes (middle) are selected based on whether the query concerns one document or a collection; the application (bottom) is the only stateful, user-facing layer.

- **Multilingual embeddings are mandatory, not incidental.** Because source documents are in French and Arabic, an English-only embedding model would silently degrade retrieval quality without raising any error (Section 6). The multilingual model `bge-m3` was adopted from Day 1 specifically to avoid this failure mode.

The practical cost of this choice is real: local models are smaller and less capable than the largest commercial offerings, and this shows most clearly in the multi-step calculation results discussed in Section 5. The trade-off is deliberate: for a document-intelligence system handling financial data, keeping inference sovereign is treated as a harder constraint than maximising raw model capability.

4 Evaluation Methodology and Results

Consistent with the project’s guiding principle — *report real numbers, not impressions* — every capability was measured against a hand-built or externally sourced gold set, rather than assessed by spot-checking.

4.1 Structured Extraction

A gold set of 12 invoices was hand-labelled (Section 7), covering deliberately varied vocabulary, formats, and missing-field cases. Table 2 and Figure 2 report field-level precision, recall, and F1 against this set.

Layer	Tool	Rationale
Digital PDF text OCR (scanned)	pdfplumber RapidOCR DeepSeek-OCR (CPU); (server, hard cases)	Exact text extraction, no GPU, instant CPU-only for the common case; heavier model reserved for difficult scans
LLM	Ollama, self-hosted (llama3.2)	Runs on our own infrastructure — no document content leaves the local/team environment
Embeddings	bge-m3 (multilingual, via Ollama)	Required for correct FR/AR retrieval (Section 3 and 6)
Vector store	FAISS (IndexFlatL2)	Exact nearest-neighbour search, appropriate at this document-collection scale
Structured extrac- tion Application	Ollama JSON mode + explicit schema Streamlit	Reliable key-value output instead of brittle regular expressions Rapid, stateless UI over cached extraction results

Table 1: Technology choices and their justification.

Field	Precision	Recall	F1
Invoice number	0.92	1.00	0.96
Date	1.00	1.00	1.00
Issuer	1.00	0.82	0.90
Client	1.00	1.00	1.00
Total	1.00	1.00	1.00

Table 2: Field-level extraction accuracy, $n = 12$ documents.

4.2 Grounded Question Answering and RAG

A second gold set (`qa_pairs.json`) was constructed with five questions per document (60 total): three direct lookups, one requiring a small arithmetic computation, and one deliberately unanswerable question intended to test the refusal guardrail. Two metrics are reported separately per question type: *retrieval hit-rate* (did FAISS return a chunk from the correct source document?) and *answer accuracy* (was the final answer correct?). Results are shown for the tuned configuration ($k = 5$; see Section 5 for the $k = 3$ comparison).

Question type	Retrieval hit-rate	Answer accuracy
Lookup	72%	42%
Calculation	92%	25%
Unanswerable (guardrail)	67%	67%

Table 3: RAG performance by question type, $n = 60$ questions, top- $k = 5$.

4.3 OCR Accuracy

RapidOCR was evaluated against five real, hand-annotated receipts from the public SROIE dataset (ICDAR 2019), which ships ground-truth labels. Two metrics are reported for text fields:

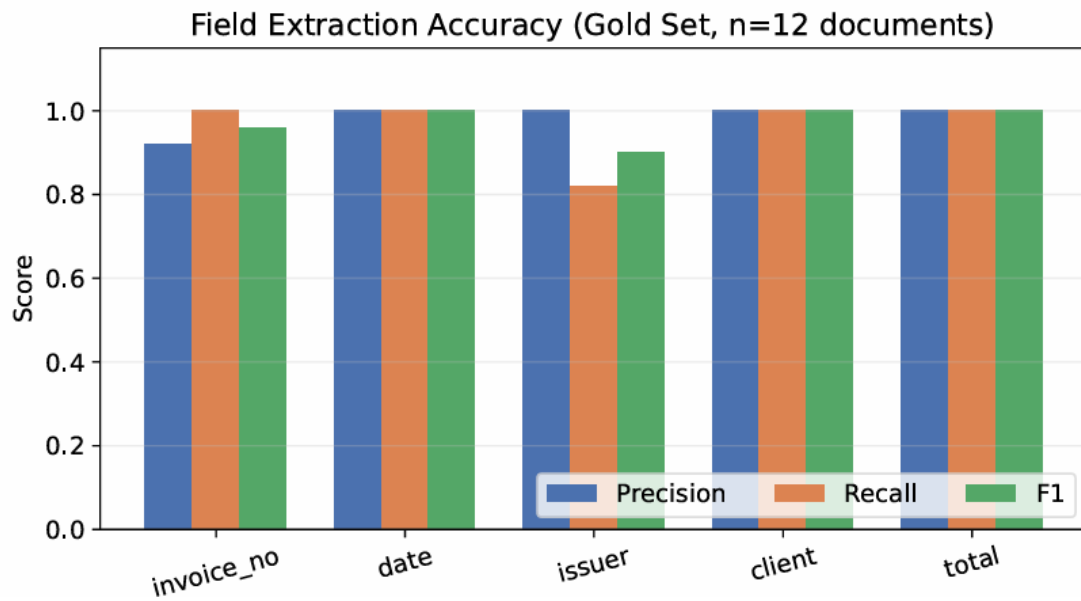


Figure 2: Extraction precision, recall and F1 by field.

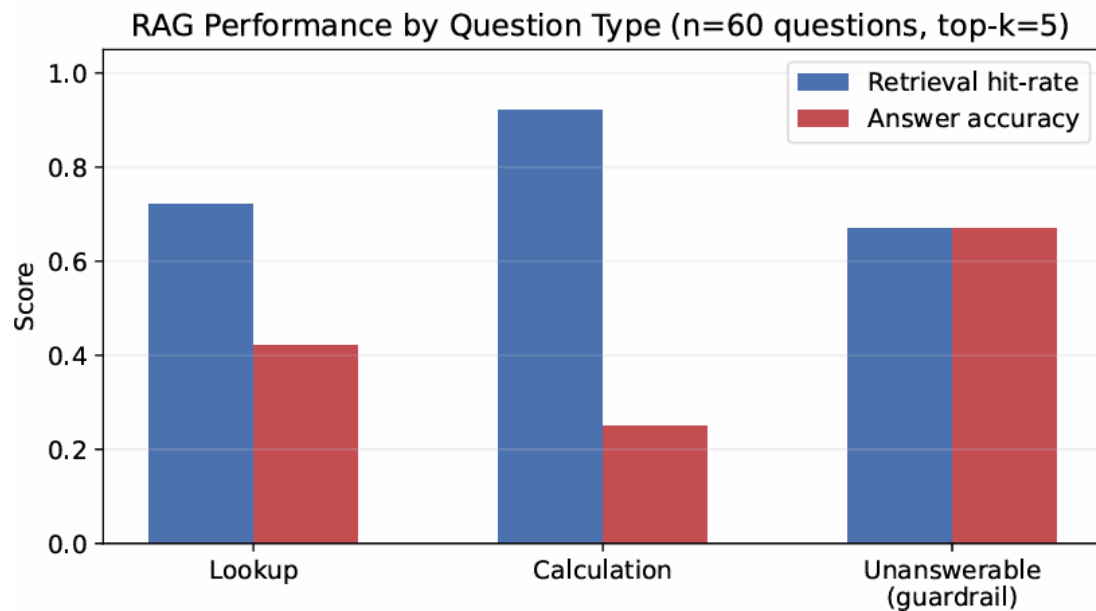


Figure 3: Retrieval hit-rate versus answer accuracy, by question type.

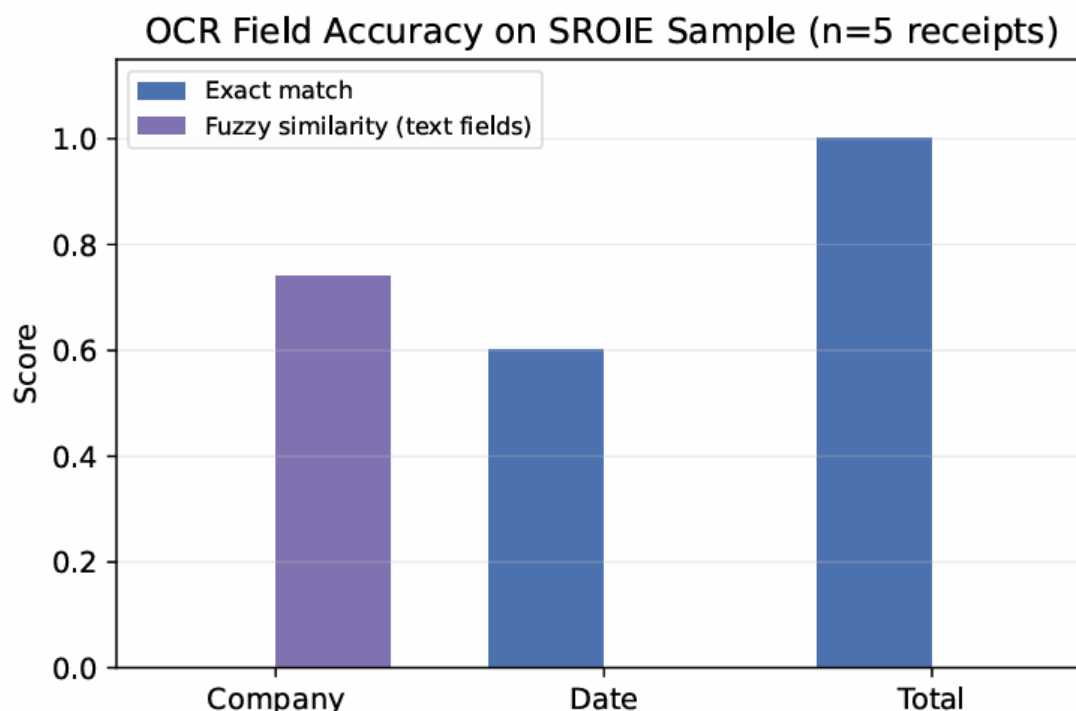


Figure 4: OCR accuracy: exact match versus fuzzy similarity for text fields.

exact string match, and a fuzzy similarity ratio (`difflib.SequenceMatcher`), since exact match alone understates near-correct OCR output.

Field	Exact match	Fuzzy similarity
Company name	0%	74%
Date	60%	—
Total	100%	— (numeric field)

Table 4: OCR field accuracy on SROIE sample, $n = 5$ receipts.

Numeric fields (totals) are extracted with perfect exact-match accuracy. Company names show 0% exact match but 74% average fuzzy similarity, indicating that failures are dominated by minor OCR segmentation noise (missing spaces, single-character confusions such as "INIAH" vs. "INDAH") rather than wholesale extraction failure. This distinction — confirmed by manually inspecting the raw OCR output for each case — materially changes the interpretation of the 0% figure and is discussed further in Section 6.

4.4 Application Interface

The pipeline is exposed through a three-tab Streamlit application. The first tab (Figure 5) lets a user upload a PDF and immediately view its extracted fields as a structured table, including detected line items. The second tab provides a chat-style question-answering panel (Figure 6) that preserves conversation history, lets the user switch between single-document and RAG mode per question, and displays the exact source citation beneath every answer. A third tab summarises the evaluation metrics from Section 4 as a live dashboard, so that a user of the tool can see its measured reliability rather than only its output.

FinDocIQ — Extraction et Q&A sur documents financiers ↔

Extraction de champs Question/Réponse Métriques du système

Téléverse un document pour extraire ses champs

Choisis un PDF

facture_test.pdf ×
1.8KB +

Extraction terminée

numero_facture	INV-2026-001
date	22/07/2026
emetteur	Atelier Bois & Cie
client	Amina Ouayy
total	1900

Lignes détaillées :

description	montant
Table en chene	1200
Chaises (lot de 4)	700

Figure 5: Field-extraction tab: a user-uploaded invoice is parsed into a structured table, with detailed line items shown separately.

4.5 Automated Financial Consistency Checks

As a final, model-independent safety net, `checks.py` verifies that extracted line-item amounts sum to the extracted total, and computes each line item's share of the total. On `facture_test.pdf`, the check confirmed $1200 + 700 = 1900$ (consistent), with the two line items representing 63.2% and 36.8% of the total respectively. This check is independent of the LLM and can flag extraction errors without requiring a human to re-read the source document.

5 Hallucination and Guardrail Discussion

5.1 Guardrail Design and Measurement

The grounding prompt instructs the model to answer strictly from the supplied context, to state explicitly when an answer is not present, and to cite the exact source snippet used. This was tested directly against the twelve deliberately unanswerable questions in the gold Q&A set.

FinDocIQ — Extraction et Q&A sur documents financiers

Extraction de champs Question/Réponse Métriques du système

Pose une question sur tes documents

Mode :

- ☒ Document unique (direct)
☐ Plusieurs documents (RAG)

Choisis un document :

factures\facture_test.pdf



Qui est l'émetteur de cette facture ?



Atelier Bois & Cie

Mode : Document unique (direct) | Citation : Emetteur : Atelier Bois & Cie

Pose ta question ici...



Quelle est la date de la facture ?



22/07/2026

Mode : Document unique (direct) | Citation : Date : 22/07/2026

Figure 6: Chat-style Q&A tab: each answer is shown with its source citation and the mode used to generate it (here, single-document direct mode).

In a single-document setting, the guardrail correctly refused 11 of 12 unanswerable questions (92%). The one failure is a genuine hallucination, not a citation-formatting issue: for a document with no issuer field, the model invented "L'État" as the issuer, with no supporting text anywhere in the source document. This confirms that even a strict prompt with temperature = 0 does not eliminate hallucination risk entirely.

In the multi-document RAG setting, the guardrail success rate measured lower, at 67%. This is consistent with the added difficulty of the RAG setting: the model must apply the same refusal judgement over several retrieved chunks from unrelated documents rather than one clean source, increasing the chance of spuriously matching irrelevant context.

5.2 Retrieval Tuning: $k = 3$ versus $k = 5$

Table 5 compares retrieval hit-rate under two values of k (the number of chunks retrieved per query). Increasing k from 3 to 5 improved retrieval hit-rate substantially for both lookup and

calculation questions, while leaving answer accuracy roughly unchanged (and slightly reducing guardrail accuracy, plausibly due to additional irrelevant context being surfaced).

Type	Retrieval $k=3$	Retrieval $k=5$	Answer $k=3$	Answer $k=5$
Lookup	58%	72%	44%	42%
Calculation	83%	92%	17%	25%
Unanswerable	58%	67%	75%	67%

Table 5: Effect of top- k on retrieval and answer quality.

5.3 The Real Bottleneck: Arithmetic Reasoning, Not Retrieval

The calculation-question results are the most informative finding of the evaluation. At $k = 5$, retrieval hit-rate for calculation questions is 92% — the correct source document is found almost every time — yet answer accuracy is only 25%. Manual inspection of failure cases (e.g., a question asking for the difference between two medication totals) showed the model correctly locating both relevant figures but performing the arithmetic incorrectly, in one case ignoring an item quantity and comparing a unit price directly against a multi-unit total.

This isolates the bottleneck precisely: it is *not* a retrieval problem, and prompt engineering aimed at improving citation formatting did not resolve it either (a targeted prompt change separating "cited passage" from "computed value" improved output cleanliness but not arithmetic correctness). It is a genuine capability limit of the small, locally-hosted language model (llama3.2, 3B parameters) on multi-step numeric reasoning, a direct, measured cost of the sovereignty constraint discussed in Section 3.

6 Limitations

- **Multi-step calculations are unreliable** (25% answer accuracy despite 92% retrieval hit-rate), as detailed in Section 5. This is the single most significant limitation of the current system.
- **DeepSeek-OCR (hard scans) was not evaluated.** It requires a dedicated GPU (16GB+ VRAM) unavailable on the development machine; RapidOCR-on-CPU results (Section 4.3) stand in as the only measured OCR baseline.
- **OCR company-name extraction is noisy** on real scanned receipts (0% exact match), though 74% fuzzy similarity shows the underlying signal is largely present; the failure mode is OCR segmentation noise rather than semantic misunderstanding.
- **Aggregation questions are out of scope.** Questions requiring reasoning across the entire document collection (e.g., "which invoice has the highest total?") are poorly served by top- k retrieval, which surfaces only the chunks most semantically similar to the question, not necessarily an exhaustive or representative sample. This is a known limitation of classic RAG, not specific to this implementation.
- **Line-item extraction was scoped conservatively.** The evaluation focused on header-level fields (issuer, date, total); line-item extraction was added in Week 4 for the financial consistency checks but was not subjected to the same field-level precision/recall evaluation as header fields.

- **LLM outputs are not fully deterministic.** Despite temperature=0, repeated runs on the same document occasionally produced different extracted values, particularly on documents with ambiguous vocabulary.

7 Gold Set Construction

Two complementary data sources were used for evaluation:

- **12 synthetically generated invoices**, covering varied vocabulary (invoice, receipt, purchase order, professional fee note), a fully English document, a two-page document, amounts written with commas, and deliberately missing fields — used for the main extraction and RAG evaluation (Tables 2–5).
- **5 real scanned receipts** from the public SROIE dataset (ICDAR 2019), which ships independently produced ground-truth labels — used for the OCR evaluation (Table 4), avoiding any risk of evaluating OCR against self-generated (and therefore artificially clean) documents.
- **3 pages from a real financial annual report** (Crédit Agricole du Maroc, 2024) were extracted as a stretch case but kept out of the primary evaluation, since their field schema (aggregate financial indicators) differs structurally from the invoice schema and mixing the two would have produced a misleading combined score.

8 Conclusion and Future Work

FinDocIQ demonstrates a working, fully self-hosted pipeline for extracting and querying financial documents, with every capability backed by a quantitative, gold-set-based evaluation rather than qualitative impression. The system performs strongly on structured field extraction ($F1 \geq 0.90$ on all fields) and on numeric consistency checks, and its guardrail meaningfully reduces (though does not eliminate) hallucination. The clearest weakness — multi-step arithmetic reasoning in the local LLM — is now precisely measured and isolated from retrieval quality, providing a concrete target for future work:

- Evaluate whether a larger locally-hosted model (e.g., an 8B-parameter variant) improves calculation accuracy while remaining sovereign.
- Complete the DeepSeek-OCR evaluation once server access is available, to justify (or revise) the hard-scan routing decision.
- Extend line-item extraction evaluation with field-level precision/recall, as was done for header fields.
- Investigate a dedicated aggregation path (e.g., pre-computed summary statistics) for whole-collection questions that current top- k retrieval cannot answer reliably.

Full source code, evaluation scripts, and gold sets are available at: <https://github.com/amina63/findociq>